DEGREE PROJECT IN TECHNOLOGY,
SECOND CYCLE, 30 CREDITS
STOCKHOLM, SWEDEN 2019

Testing Resilience of Envoy Service Proxy with Microservices

Nikhil Dattatreya Nadig

KTH ROYAL INSTITUTE OF TECHNOLOGY
ELECTRICAL ENGINEERING AND COMPUTER SCIENCE

Abstract

Large scale internet services are increasingly implemented as distributed systems to achieve availability, fault tolerance and scalability. To achieve fault tolerance, microservices need to employ different resilience mechanisms such as automatic retries, rate limiting, circuit breaking amongst others to make the services handle failures gracefully and cause minimum damage to the performance of the overall system. These features are provided by service proxies such as Envoy which is deployed as a sidecar (sidecar proxy is an application design pattern which abstracts certain features, such as inter-service communications, monitoring and security, away from the main architecture to ease the tracking and maintenance of the application as a whole), the service proxies are very new developments and are constantly evolving. Evaluating their presence in a system to see if they add latency to the system and understand the advantages provided is crucial in determining their fit in a large scale system.

Using an experimental approach, the services are load tested with and without the sidecar proxies in different configurations to determine if the usage of Envoy added latency and if its advantages overshadow its disadvantages. The Envoy sidecar proxy adds latency to the system; however, the benefits it brings in terms of resilience, make the services perform better when there is a high number of failures in the system.

Keywords

Service Proxy, Distributed Systems, Resilience Patterns, Automatic Retries, Rate Limiting, Circuit Breakers

Abstract

Storskaliga internettjänster implementeras alltmer som distribuerade system för att uppnå tillgänglighet, feltolerans och skalbarhet. För att uppnå feltolerans måste microservices använda olika typer av resiliens mekanismer som automatisk återförsök, hastighetsbegränsning, kretsbrytning bland annat som tillåter tjänsterna att hantera misslyckanden graciöst och orsaka minimala skador på prestandan hos det övergripande systemet. Dessa funktioner tillhandahålls av service proxies som Envoy. Dessa proxies används som sidovagn (sidvagnproxy är ett applikationsdesignmönster som abstraherar vissa funktioner, såsom kommunikation mellan kommunikationstjänster, övervakning och säkerhet, bort från huvudarkitekturen för att underlätta spårningen och underhåll av ansökan som helhet). Dessa tjänster är väldigt nya och utvecklas ständigt. Att utvärdera deras närvaro i ett system för att se om de lägger till latens för systemet och förstå fördelarna som tillhandahålls är avgörande för att bestämma hur väl de skulle passa i ett storskaligt system. Med hjälp av ett experimentellt tillvägagångssätt testas tjänsterna med och utan sidospårproxys i olika konfigurationer för att avgöra om användningen av Envoy lägger till latens och om dess fördelar överskuggar dess nackdelar. Envoy sidecar proxy ökar latensen i systemet; De fördelar som det ger med avseende på resiliens gör tjänsterna bättre när det finns ett stort antal misslyckanden i systemet.

Nyckelord

Service Proxy, Distributed Systems, Resilience Patterns, Automatic Retries, Rate Limiting, Circuit Breakers

Acknowledgements

I would like to thank my industrial supervisor Florian Biesinger for his unwavering interest, support and patience to mentor me throughout the thesis, Henry Bogaeus for his constant willingness to help, assistance in keeping my progress on schedule and to discuss new ideas with, and all of my colleagues with whom I had the pleasure of working with.

I want to thank my KTH examiner, Prof. Seif Haridi and my supervisor Lars Kroll for providing me with valuable inputs and guidance throughout the thesis. I am grateful to all my friends for their support and encouragement all along.

Finally, my deep and sincere gratitude to my family for their unconditional love, help and support. I am forever indebted to my parents for giving me the opportunities and enabling me to pursue my interests. This would not have been possible if not for them.

Authors

Nikhil Dattatreya Nadig <nadig@kth.se>
Information and Communication Technology
KTH Royal Institute of Technology

Place for Project

Stockholm, Sweden
Spotify AB

Examiner

Seif Haridi
Kista, Sweden
KTH Royal Institute of Technology

Supervisor

Florian Biesinger
Berlin, Germany
Spotify GmbH

Lars Kroll
Kista, Sweden
KTH Royal Institute of Technology

Contents

1 Introduction

Large scale internet services are being implemented as distributed systems to achieve fault tolerance, availability, and scalability. This is evident by a shift from monolithic architectures towards microservices. A monolithic architecture such as three-tier architecture provides maintainability where each tier is independent, updates or changes can be carried out without affecting the application as a whole. Since there is a high level modularity, it is possible to have a rapid development of the application. However, with the advent of smart devices, where applications need to be available on more than one platform three tier architecture would be difficult to build and cumbersome to maintain. By using microservices, applications can be built for multiple platforms.

1.1 Context and Motivation

It is evident from the introduction that microservices architectures remove complexity from the code by making it single purpose but increase the complexity of operations. Microservices utilise APIs at their endpoints, one of its key features is the ability for services to replicate throughout the system as required. Since the application structure is continuously changing, features such as automatic retries, rate limiting, circuit breaking and other system level features that help the systems work better at scale need not be present in the service itself. Service Proxies deployed as sidecars to the microservices not only have these features but also provide service discovery, health checking, advanced load balancing, HTTP routing and distributed tracing.

However, adding a sidecar proxy to each of the microservices means that instead of a message being sent from one service to other, there are two more services i.e, the message needs to go two intermediate services which are the proxies themselves. If messages need to go through more services to reach its destination, there is a chance for some amount of added latency induced by this. If the latency is big and the application consists of dozens of microservices each having a sidecar proxy, the combined latency of the system can increase to extent that the merits of the sidecar proxy is overshadowed by its demerits.

1.2 Problem Statement and Research Question

Envoy service proxy[10] provides various benefits such as service discovery, health checking, advanced load balancing and HTTP routing along with resiliency to services with automatic retries, rate limiting and circuit breaking. However, due to the presence of additional services there is a possibility of the latency of the services increasing when operating at scale. This might go against the design goals of application and might cause unexpected problems to the system.

This research provides understanding if the presence of a service proxy deployed as a sidecar to microservices added additional latency and determine if the resiliency patterns offered by the service proxies improved the performance of the entire system. Additionally, by changing error probabilities and introducing latency within the system, we study the affects of these with and without a service proxy.

This research helps understand the role of service proxy(developed as a sidecar to microservices) in additional latency. It also helps determine if the resiliency patterns offered by these service proxies helped improve the performance of the entire system.

1.3 Purpose

The purpose of this thesis is to determine if the use of a service proxy deployed as a sidecar in a microservices system induces substantial latency and reduces the performance of the overall system to the extent that a system without sidecar proxies performs a better than a system deployed with sidecar proxies. Testing the resilience capabilities such as automatic retries, rate limiting, and circuit breaking provided by the service proxy which help the services handle failures better and compare the results with the test run without using Envoy to understand the advantages of Envoy.

1.4 Goal

The goal of this dissertation is to perform load testing on a group of microservices that make up a system, each of which have a sidecar proxy deployed with the services, measure its performance and determine if there is latency caused by the addition of the service proxy. As part of load testing we tested the resiliency features provided by the Envoy service proxy to see how it the automatic retries, rate limiting and circuit breaking and determine if it helped the system to recover from errors, thereby improving the overall performance of the system.
The approach for conducting this dissertation involved four weeks of research to find the various options available for improving resiliency in distributed systems while operating at scale, understanding their features and determining which fit the requirements the best. The next two weeks involved building prototypes of systems using various services meshes to determine its ease of use.

1.5 Methodology

This research was conducted in an experimental approach. For a collection of microservices with service proxies deployed as a sidecar which communicate over gRPC[15], determining if there was added latency and comparing how latency in the system affects the entire system with and without a service proxy. Parameters such as requests per second, number of errors thrown by each experiment were used to perform analysis. Since the microservices were written using Java, the data from the first experiment was discarded to avoid latency caused by the JVM to affect the experiment results. Additionally, testing the resiliency features offered by the Envoy service proxy to determine if they add latency and reduce the overall performance. This was performed in a manner that the results could be reproducible.

1.6 Delimitations

Service proxies are a very young field. There is not much academic research about this topic, and most of the literature study for these have been in the form of the

official documentation, books, and blog posts.

Service proxies and Envoy in particular is used as sidecar proxies along with service meshes such as Istio[18], Consul[7], Linkerd2[22] which use Kubernetes[26] as a container-orchestration system. Using all of these components to perform testing can become complex as the possible configuration space to explore may become too large to be tractable. So to limit the scope of this research, service meshes and kubernetes was not used in the experiment setup. Additionally, features of Envoy proxy such as distributed tracing, advanced load balancing was not evaluated.

1.7 Outline

Chapter 2 gives a background about the evolution of cloud computing, microservices, resiliency patterns that can be introduced to microservices, gRPC framework and other related work used around service proxies such as service meshes and kubernetes. Chapter 3 is mainly describes the implementation of the services, the services setup, load testing client, and the design of the experiments. Chapter 4 contains the experiment results along with the analysis of the results gathered throughout the experiments. Chapter 5 discusses the future work and conclusion of the dissertation.

2 Background

In this chapter, we start with an overview of the evolution of cloud computing. This is followed by a section describing different resiliency patterns used in microservices. Key concepts and systems that are used in the industry surrounding service proxies is presented as related work along with some underlying concepts for context.

2.1 Evolution of Cloud Computing

This section is a brief introduction to the evolution of cloud computing technologies. It provides an industrial outlook on how architectures and paradigms evolved over the years to achieve a distributed system. As part of the literature study, this section introduces some concepts of the infrastructure used throughout the rest of the dissertation.

2.1.1 N - Tiered Architecture

Most internet services followed a three-tier architecture until recently. The three layers included the data layer, business layer, and application layer. These tiers did not necessarily correspond to the physical locations of the various computers on a network, but rather to logical layers of the application.[4] The data layer primarily interacted with persistent storage, usually a database. The business layer accessed this layer to retrieve data to pass onto the presentation layer. The business layer was responsible for holding the logic that performed the computation of the application and essentially moving data back and forth between the other two layers. The application layer was responsible for all the user-facing aspects of the application. Information retrieved from the data layer was presented to the user in the form of a web page. Web-based applications contained most of the data manipulation features that traditional applications used.

2.1.2 Service Oriented Architecture

Service Oriented Architecture(SOA) is a software architecture pattern which the application components provide services to other components via a communications protocol over network[31]. The communication can involve a simple data passing or could involve two or more services coordinating connecting services to each other. Services carry out small functions such as activating an account or confirming orders. There are two main roles in an SOA - a service provider and a service consumer. The consumer module is the point where consumers (end users, other services or third parties) interact with the SOA and Provider Layer consists of all the services defined within the SOA.

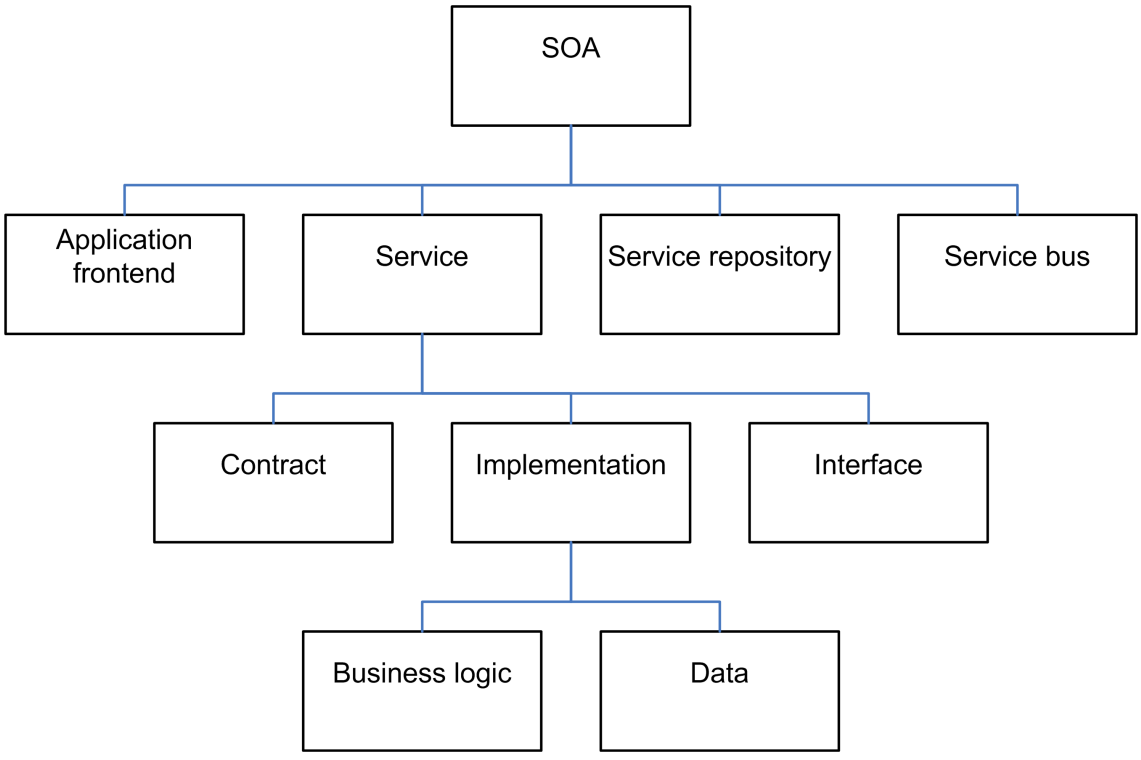

Figure 2.1: Service Oriented Architecture.[19]

2.1.3 Microservices Architecture

Although 3 tier architecture was previously used to create an application, there are several problems with this type of system. All code is present within the same code base and deployed as a single unit. Even though they are logically separated as modules, they are dependent on one another and runs as a single process

context. Any change requires the entire application to be built and re-deployed. This does not scale well and is problematic to maintain. Microservices are a software development technique—a variant of the service-oriented architecture architectural style that structures an application as a collection of loosely coupled services. In a microservices architecture, services are fine-grained and the protocols are lightweight. The benefit of dividing an application into different smaller services is that it improves modularity. This makes the application easier to understand, develop, test, and deploy[25]. Figure 2.2 depicts the difference between a microservices architecture and a monolithic or a three-tiered architecture.

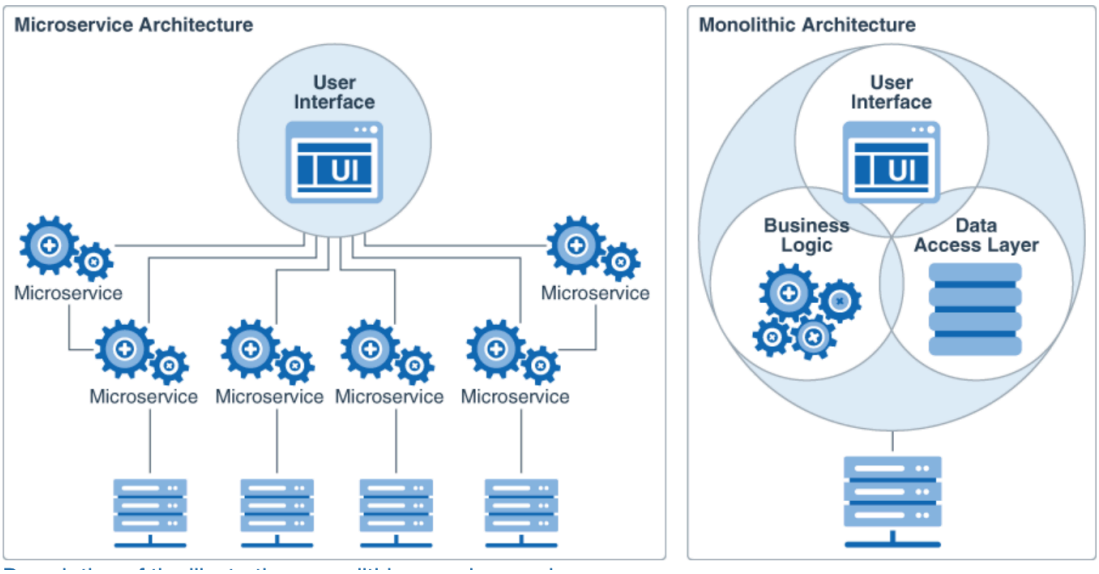

Figure 2.2: Microservices Architecture v/s Monolithic Architecture[20].

2.1.4 Virtual Machines

Virtual machines is defined as ”an efficient, isolated duplicate of a real computer machine” [3] There are different kinds of virtual machines, each of which serve a different functions:

System Virtual Machines They are also known as full virtualisation virtual machines which provide a substitute for a real machine, i.e. provide the functionality needed to execute an entire operating system. A hypervisor is a

software that creates and runs virtual machines. It uses native execution to share and manage hardware which allows multiple environments to coexist in isolation while sharing the same physical machine[30].

Process Virtual Machines They are also called as application virtual machines which are designed to execute computer programs in a platform-independent environment, i.e run as a normal application inside a host OS and supports a single process. The VM is created when the process is created and destroyed when the process exits.

Virtual machines provide the ability to run multiple operating system environments on the same computer. VMs also provide security benefits by providing a "quarantine" workspace to test questionable applications and perform security forensics by monitoring guest operating systems for deficiencies and allowing the user to quarantine it for analysis.

However, virtual machines are less efficient than real machines since they access the hardware indirectly, i.e. running software on a host OS which needs to request access to the hardware from the host. When there are several VMs running on a single host, performance of each of the VMs is hindered due to the lack of resources available to each of the VMs.

2.1.5 Docker Containers

Docker is a platform for developing, shipping, and running applications[34]. Docker is used to run packages called "containers". Containers are isolated from each other and bundle their application, tools, libraries and configuration files. Containers can communicate with each other through well-defined channels. All containers run on a single operating system kernel and are thus more lightweight than virtual machines. Docker enables the separation of applications from the infrastructure. This helps in the management of infrastructure in the same way as the management of an application. Hence, reducing the delay between implementation and deployment of the application in production.

2.1.6 Comparison of Docker Containers and Virtual Machines

There are use cases when either containers or virtual machines is the better choice to make. Virtual machines are a better choice when applications running need access to the complete operating system's functionalities, resources or need to run several applications in one server. On the other hand, containers are better suited when the goal is optimal resource utilization, i.e, to run the maximum number of applications on a minimum number of servers.

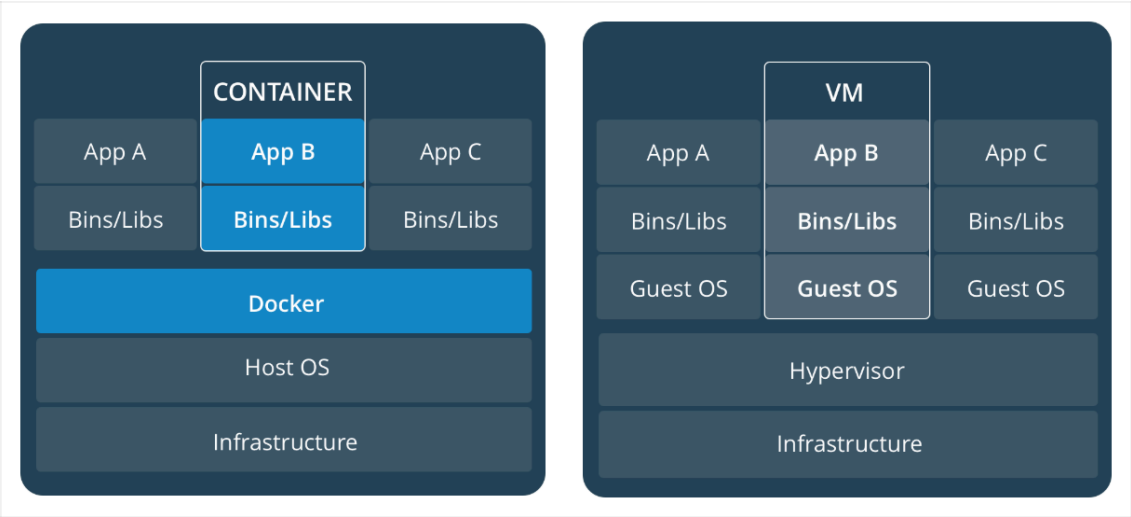

Figure 2.3: Docker v/s VMs[13].

2.1.7 Issues with Microservices

Despite all the advantages Microservices provide over a traditional monolithic application, several trade-offs are made to use microservices. In a monolithic architecture, the complexity and the dependencies are inside a single code base, while in microservices architectures complexity moves to the interactions of the individual services that implement a specific function. Challenges like asynchronous communication, cascading failures, data consistency problems, and authentication of services are essential to a successful microservices implementation. Microservices has not only increased the technical complexity of the system but has increased the operational complexity of running and maintaining the system. Problems such as monitoring the overall health of a

system, request tracing, tracking, fault tolerance, and debugging interactions across the entire systems are some of the issues faced while maintaining a microservices architecture based system.

When there is a complex system in a distributed environment, it is only a matter of time before some systems will fail. Failure could be due to latency, upstream error or several other reasons. If the host application is not isolated from these external failures, it risks being taken down with them. With high volume traffic a single backend dependency becoming latent can cause all resources to become saturated on all servers. These can result in increased latencies between services, which increases message queues, threads, and other system resources causing even more cascading failures across the system. Network connections degrade over time, services and servers become fail or become slow. All these represent failures that need to be managed so that a single dependency does not crash an entire system.

Hystrix Hystrix was created by Netflix[23] to prevent all of this by isolating points of access between the services, stopping cascading failures across them, and providing fallback options, which improve the system's overall resiliency.
Bulkheading is the process of isolating elements of an application into pools so that if one fails, the others will continue to function. Partitioning service instances into several groups, based on load and availability requirements helps in isolating failures and allows to sustain service functionality for some consumers, even during a crash.
Hystrix' has two different approaches to the bulkhead:

- Thread Isolation: The standard approach is to hand over all requests to a separate thread pool with a fixed number of threads and no request queue.
- Semaphore Isolation: This approach is to have all callers acquire a permit before sending a request. If a permit is not acquired from the semaphore, calls to the service are not passed through.

2.1.8 Service Proxy

Envoy Envoy is an L7 (detailed description of the OSI model is presented in Section 2.3.1) proxy and a communication bus designed for large scale service-oriented architectures. It is a self-contained process that runs alongside every application server. All of the sidecars form a communication mesh from which each application sends and receives messages to and from localhost and is unaware of the network topology.

Out of process architecture: Envoy is a self-contained process designed to run alongside every application server. Envoy forms a transparent communication mesh where each application sends and receives messages to and from localhost and is unaware of the network topology. The out of process architecture has two considerable advantage over the traditional library approach to service to service communication:

- Envoy works with any application language. Since it is becoming increasingly common for service-oriented architectures to use multiple application frameworks, a single Envoy deployment can form a mesh between Java, C++, Go, PHP, Python, and many other languages.
- Envoy can be deployed and upgraded quickly across an entire infrastructure transparently.

Modern C++11 code base: Envoy is written in C++11. Application developers already deal with tail latencies that are difficult to reason about due to deployments in shared cloud environments and the use of very productive but not particularly well-performing languages such as PHP, Python, Ruby, Scala. Native code provides excellent latency properties, and unlike other native code proxy solutions written in C, C++11 provides both superior developer productivity and performance.

L3/L4 filter architecture: At its core, Envoy is an L3/L4 network proxy. A pluggable filter chain mechanism allows filters to be written to perform different TCP proxy tasks and inserted into the central server. Filters have already been written to support various tasks such as raw TCP proxy, HTTP proxy, and TLS client certificate authentication.

HTTP L7 filter architecture: HTTP is a crucial component of modern application architectures that Envoy supports an additional HTTP L7 filter layer. HTTP filters are plugged into the HTTP connection management subsystem that performs different tasks such as buffering, rate limiting, routing/forwarding.

First class HTTP/2 support: When operating in HTTP mode, Envoy supports both HTTP/1.1 and HTTP/2. Envoy operates as a transparent HTTP/1.1 to HTTP/2 proxy in both directions. Any combination of HTTP/1.1 and HTTP/2 clients and target servers can be bridged.

HTTP L7 routing: When operating in HTTP mode, Envoy supports a routing subsystem that is capable of routing and redirecting requests based on path, authority, content type, runtime values. This functionality is most useful when using Envoy as a front/edge proxy but is also used when building a service to service mesh.

gRPC support: gRPC is an open-source RPC framework from Google that uses HTTP/2 as the underlying multiplexed transport. Envoy supports all of the HTTP/2 features required as the routing and load balancing substrate for gRPC requests and responses. The two systems are complementary.

Service discovery and dynamic configuration: Envoy consumes a layered set of dynamic configuration APIs for centralized management. The layers provide an Envoy with dynamic updates about hosts within a backend cluster, the backend clusters themselves, HTTP routing, listening sockets, and cryptographic material. For a more straightforward deployment, backend host discovery is made through DNS resolution, with the further layers replaced by static config files.

Health checking: Service discovery in Envoy is an eventually consistent process. Envoy includes a health checking subsystem which can optionally perform active health checking of upstream service clusters. Envoy then uses the union of service discovery and health checking information to determine healthy load balancing targets. Envoy also supports passive health checking via an outlier detection subsystem.

Advanced load balancing: Load balancing among different components in a distributed system is a complex problem. Since Envoy is a self-contained proxy

instead of a library, it can implement advanced load balancing techniques in a single place and be accessible to any application. Currently, Envoy includes support for automatic retries, circuit breaking, global rate limiting via an external rate limiting service, request shadowing, and outlier detection. Future support is planned for request racing.

Front/edge proxy support: Although Envoy is designed as a service-to-service communication system, there is a benefit in using the same software at the edge (observability, management, identical service discovery and load balancing algorithms). Envoy includes features to make it usable as an edge proxy for most modern web application use cases including TLS termination, HTTP/1.1 and HTTP/2 support, as well as HTTP L7 routing.

NGINX Nginx is a web server which can also be used as a reverse proxy, load balancer, mail proxy and HTTP cache.[24] It supports serving static content, HTTP L7 reverse proxy load balancing, HTTP/2, and many other features. Envoy provides the following main advantages over nginx as an edge proxy:

- Full HTTP/2 transparent proxy. Envoy supports HTTP/2 for both downstream and upstream communication. nginx only supports HTTP/2 for downstream connections.
- Ability to run the same software at the edge as well as on each service node. Many infrastructures run a mix of nginx and HAProxy. A single proxy solution at every hop is substantially simpler from an operations perspective.

HAProxy HAProxy stands for High Availability Proxy, it is a TCP/HTTP based load balancer.[16] It distributes a workload across a set of servers to maximise performance and optimize resource usage. HAProxy built with customizable health checks methods, allowing a number of services to be load balanced in a single running instance. However, Envoy provides the following main advantages over HAProxy as a load balancer:

- HTTP/2 support.
- Pluggable architecture.

- Integration with a remote service discovery service.
- Integration with a remote global rate limiting service.
- Substantially more detailed statistics.

AWS Elastic Load Balancer AWS Elastic Load Balancer was created by Amazon.[8] Elastic Load Balancing automatically distributes incoming application traffic across multiple targets, such as Amazon EC2 instances, containers, IP addresses, and Lambda functions. It can handle the varying loads of application traffic in a single Availability Zone or across multiple Availability Zones. Elastic Load Balancing offers three types of load balancers that all feature the high availability, automatic scaling, and robust security necessary to make applications fault tolerant[9]. Envoy provides the following main advantages of ELB as a load balancer and service discovery system:

- Statistics and logging (CloudWatch statistics are delayed and extremely lacking in detail, logs must be retrieved from S3 and have a fixed format).
- Advanced load balancing and direct connection between nodes. An Envoy mesh avoids an additional network hop via variably performing elastic hardware. The load balancer can make better decisions and gather more interesting statistics based on zone, canary status, etc. The load balancer also supports advanced features such as retry.

SmartStack SmartStack created by AirBnB provides additional service discovery and health checking support on top of haproxy.[29] SmartStack has most of the same goals as Envoy, i.e. out of process architecture, application platform agnostic. However, Envoy provides integrated service discovery and active health checking. Envoy includes everything in a single high-performance package.

Finagle Finagle is an extensible RPC system for the JVM, used to construct high-concurrency servers.[12] Finagle implements uniform client and server APIs for several protocols, and is designed for high performance and concurrency.

Twitter's infrastructure team maintains finagle. It has many of the same features as Envoy, such as service discovery, load balancing, and filters. Envoy provides the following main advantages over Finagle as a load balancer and service discovery package:

- Eventually consistent service discovery via distributed active health checking.
- Out of process and application agnostic architecture. Envoy works with any application stack.

Linkerd Linkerd is a standalone, open source RPC routing proxy built on Netty and Finagle (Scala/JVM)[21]. Linkerd offers many of Finagle's features, including latency-aware load balancing, connection pooling, circuit-breaking, retry budgets, deadlines, tracing, fine-grained instrumentation, and a traffic routing layer for request-level routing. Linkerd's memory and CPU requirements are significantly higher than Envoy's. In contrast to Envoy, linkerd provides a minimalist configuration language and explicitly does not support hot reloads, relying instead on dynamic provisioning and service abstractions.

2.2 Resiliency Patterns in Microservices

2.2.1 Introduction

Since the increase in popularity of Microservices, companies such as Netflix and Amazon have been successful early adopters in the setting of large-scale distributed systems. However, the adoption of microservices comes with issues that seep directly from distributed systems. Some of the issues are as follows:

- Microservices rely on message passing which can lead to communication failures when there are requests in high volume
- Services may be overloaded with too many concurrent requests and resources tied up to existing requests which are waiting for responses from

other services, this can cause cascading failures

- Microservices are optimised for being deployed in the cloud and they can be relocated to different machines during runtime

2.2.2 Retry Pattern

When an application can handle transient failures which it tries to connect to a service, by retrying a failed request, the stability of the application is improved.

Problem Distributed cloud-native systems are sensitive to transient faults. These include momentary loss in network connectivity to other services, temporary unavailability of service or timeouts that could occur when a service is busy. Generally, these faults are self-correcting, if the request is re-triggered after a certain delay, it is likely to be successful.

Solution Transient failures are common in an application with the microservices architecture and should be designed to handle failures and minimise the effects of failure have on the tasks the application performs.
Following are strategies an application can use to handle request failures:

- **Cancel**: If there is a fault where the fault is not transient or not likely to succeed on retry, the application should cancel that operation and report an exception. An example of this is, authentication failure caused by invalid credentials.
- **Retry**: If the fault that occurs is rare or unusual, might have been caused due to a corrupt network packet being transmitted. In this case, the likelihood of the packet being corrupt is low and retrying the request would probably yield a correct response.
- **Retry After Delay**: If the fault is caused by connectivity, the network or service might need a short period while the connectivity issues are corrected

or the backlog of work is cleared. The application should wait for a suitable time before retrying the request.

For services that suffer from transient failures often, the duration between retries needs to be chosen to spread requests from multiple instances of the application as evenly as possible. This reduces the chances of a busy service being continuously overloaded. If the services are not allowed to recover due to retries, the service would take longer to recover.

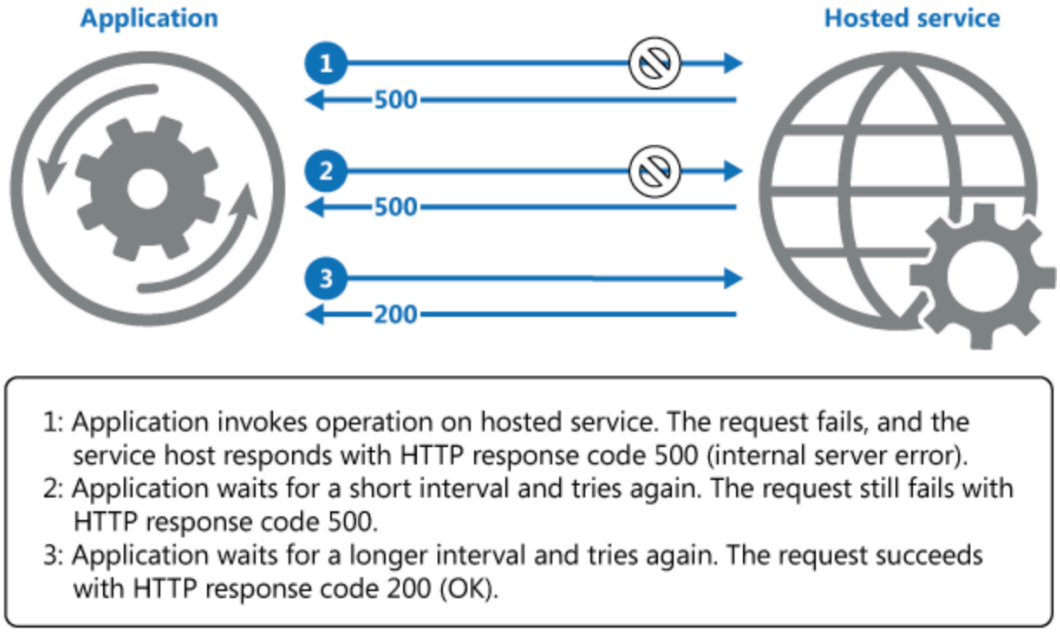

Figure 2.4: Retry Pattern[28].

2.2.3 Circuit Breaking

Problem In a distributed environment, calls to remote services can fail due to transient failures, such as slow network connections, timeouts, or the resources being over-committed or unavailable. These failures are usually self-correcting after a short duration of time.

However, there can be situations where failures occur due to unusual events that could take longer to fix. These failures can vary from a simple loss in connection to a complete failure of service. In such scenarios, there is no point for an application

to retry an operation which is not likely to succeed. Instead, the application should accept the failure of the operation and handle it accordingly.

Solution Allowing the service to continue without waiting for the fault to be fixed while it determines that the fault is long lasting. The Circuit Breaker pattern also enables an application to detect whether the fault has been resolved. If the problem appears to have been fixed, the application can try to invoke the operation.
A circuit breaker acts as a proxy for operations that might fail. The proxy should monitor the number of recent failures that have occurred, and decide if the operation should be allowed to proceed or return an exception. The proxy can be implemented as a state machine with the following states that mimic the functionality of a circuit breaker:

- Closed: The request from the service is routed to the operation and the proxy maintains the number of recent failures and if there is a failure, the proxy increments the count. If the number of failures goes beyond a certain threshold in a given period of time, the proxy is set to an OPEN state. When the proxy is in this state, the proxy times out and when the timeout expires, the proxy is placed in a HALF OPEN state
- Open: The request from the service fails immediately and an exception is returned to the service.
- Half-Open: The number of requests allowed to the pass-through and call operation is throttled. If these requests are successful, it is assumed that the failure that was caused previously is no longer causing further failures and the circuit breaker is set to the CLOSED state. If requests fail again, the circuit breaker reverts to OPEN state and gives the application more time to resolve the failure.

2.2.4 Rate Limiting

Problem In microservice architectures, resources without constraints on their usage can easily become overwhelmed by the number of clients making requests.

There may be a number of clients, each implementing various types of retry or rate-limiting policies. Such clients can easily starve resources from other clients by saturating a service and may continue to do so until it completely brings down a service.

Solution To avoid such abuse of the service, rate limiting is enabled. Envoy allows for reliable global rate limiting at the HTTP layer as compared to IP based rate limiting or an application level rate limiting like may web frameworks provide.

2.3 Related Work

2.3.1 OSI Model

The Open Systems Interconnection model (OSI model) is a conceptual model that characterises and standardises the communication functions of a computing system without regard to its underlying internal structure and technology. Interoperability of different communication systems with standard protocols is the goal. The model partitions a communication system into abstraction layers.

Physical Layer (Layer 1)

The first layer of the OSI reference model is the physical layer. It is responsible for the physical connection between devices. The physical layer contains information in the form of bits. It is accountable for the physical connection between the devices. While receiving data, this layer gets the signal received and convert it into a binary format and send it to the Data Link layer, which puts the frame back together.

Data Link Layer (DLL) (Layer 2)

The data link layer is responsible for the node-to-node delivery of the message. The primary function of this layer is to ensure data transfer over the physical layer is error free from one node to another. When a packet arrives in a network, it is the responsibility of Data Link Layer to transmit it to the Host using its MAC address.

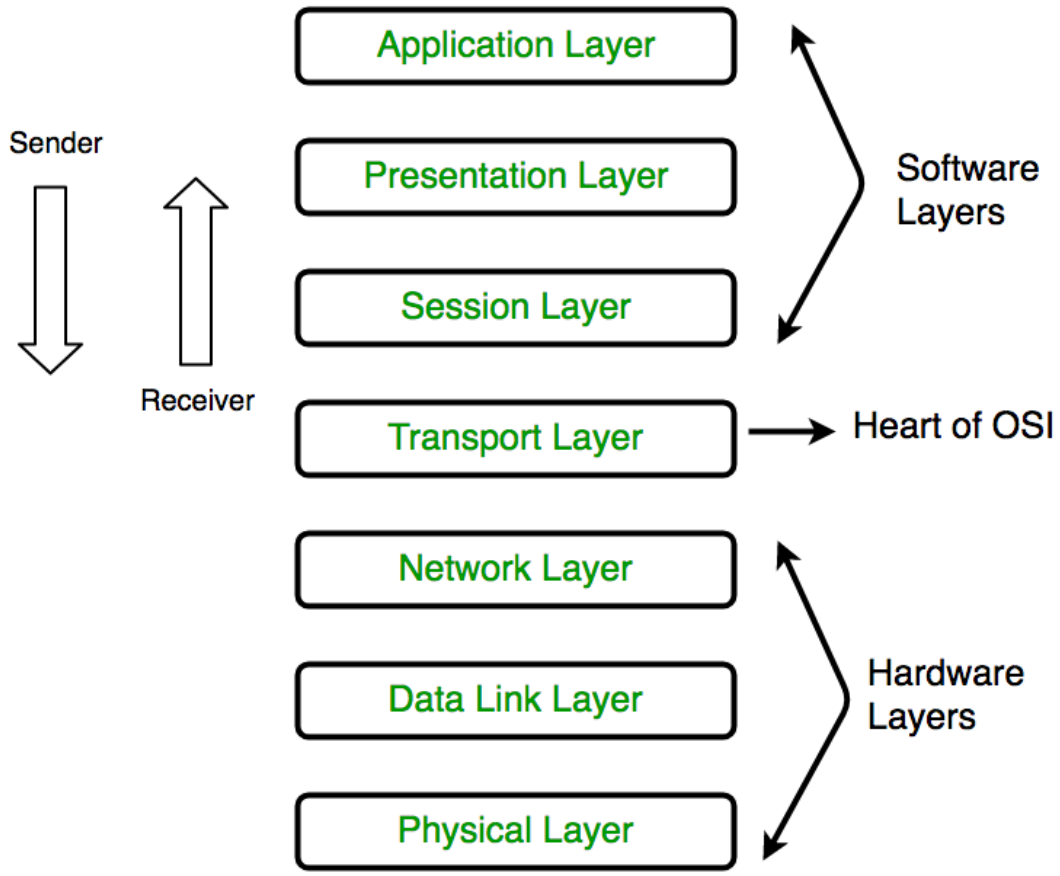

Figure 2.5: OSI Model[6].

Data Link Layer is divided into two sub layers :

- Logical Link Control (LLC)
- Media Access Control (MAC)

Network Layer (Layer 3)

Network layer performs the transmission of data from one host to the other located in different networks. It is also responsible for packet routing, i.e. selection of the shortest path to transmit the packet, from the different routes available. The sender and receiver's IP address are placed in the header by the network layer. The functions of the Network layer are :

- Routing: The network layer protocols determines the route suitable from source to destination. This function of the network layer is known as routing.
- Logical Addressing: In order to identify each device on internet work

uniquely, network layer defines an addressing scheme. The sender and receiver's IP address are placed in the header by network layer. Such an address distinguishes each device uniquely and universally.

Transport Layer (Layer 4)

The transport layer provides services to the application layer and takes services from the network layer. The data in the transport layer is called Segments. It is responsible for the end-to-end delivery of the complete message. The transport layer also provides the acknowledgement of the successful data transmission and re-transmits the data on the occurrence of an error.

At sender's side:

Transport layer receives the formatted data from the upper layers, performs Segmentation along with Flow and Error control to guarantee proper data transmission. It also adds Source and Destination port number in its header and forwards the segmented data to the Network Layer. The destination port number is generally configured, either by default or manually.

At receiver's side:

Transport Layer reads the port number from its header and forwards the data received to the respective application. It performs sequencing and reassembling of the segmented data.

The functions of the transport layer are :

- Segmentation and Reassembly: This layer receives the message from the session layer, breaks the message into smaller blocks. Each of the segment produced has a header associated with it. The transport layer at the destination reassembles the message.
- Service Point Addressing: In order to deliver the message to the correct process, the transport layer header includes a type of address called service point address or port address. By specifying the address, transport layer ensures that the message is delivered to the correct process.

Session Layer (Layer 5)

This layer is responsible for establishing a connection, maintaining a session, authentication and ensuring security. The functions of the session layer are :

- Session establishment, maintenance and termination: The layer allows the two processes to establish, manage and terminate a connection.
- Synchronisation: This layer allows a process to add checkpoints, which are considered to be synchronisation points into the data. These synchronisation point help to identify the error so that the data is re-synchronised accurately, ends of the messages are not cut prematurely and to avoid any data loss.
- Dialog Controller : The session layer allows two systems to start communication with each other either in half-duplex or full-duplex.

Presentation Layer (Layer 6)

The presentation layer is also known as the Translation Layer. The data from the application layer is extracted and formatted as per the required format to transmit over the network. The functions of the presentation layer are :

- Translation : For example, ASCII to EBCDIC.
- Encryption/ Decryption : Data encryption translates the data from one form to another. The encrypted data is known as the cypher text, and the decrypted data is the plain text. A key value is used to encrypt and decrypt the data.
- Compression: Reduces the total number of bits that need to be transmitted on the network.

Application Layer (Layer 7)

At the top of the OSI Reference Model stack of layers is the Application layer, which is implemented by the network applications. The applications generate the data, which is transferred over the network. This layer also serves as a window for applications to access the network and for displaying the received information to the user.

2.3.2 Containers Orchestration

When there are several containers deployed in a server, there is a lot of operational work that needs to be carried out such as, if the containers need to be shut down

and respawned when there are issues or deploying additional instances when there is more traffic in the system. System resource management is an essential aspect of managing servers and to use all the resources to reduce operational costs optimally. To perform all this manually is not only arduous but unnecessary. Container orchestrators such as Kubernetes can be used instead for this purpose. Kubernetes is an open-source container orchestration system for automating application deployment, scaling, and management. Google originally designed it as Borg[32] and later open-sourced it. It provides a platform for automating deployment, scaling, and operations of application containers across clusters of hosts. Containers deployed onto hosts are usually in replicated groups. When deploying a new container into a cluster, the container orchestration tool schedules the deployment and finds the most appropriate host to place the container based on predefined constraints (for example, CPU or memory availability). Once the container is running on the host, the orchestration tool manages its lifecycle as per the specifications laid out in the container's definition file, for example, a dockerfile.

As seen in figure 2.6, there are several components that make up a kubernetes orchestrator. Following are the components with their functions:

- **Kubernetes Master Component**: The Master component provides the cluster's control plane. Master component make global decisions about the cluster such as scheduling, detecting and responding to cluster events. Master component can be run on any machine in the cluster.
  - **kube-apiserver**: Component on the master that exposes the Kubernetes API. It is the front-end for the Kubernetes control plane. It is designed to scale horizontally – that is, it scales by deploying more instances.
  - **etcd**: It is a consistent and highly-available key value store used as the backing store for all cluster data.
  - **kube-scheduler**:
    It is a component on the master that monitors newly created pods

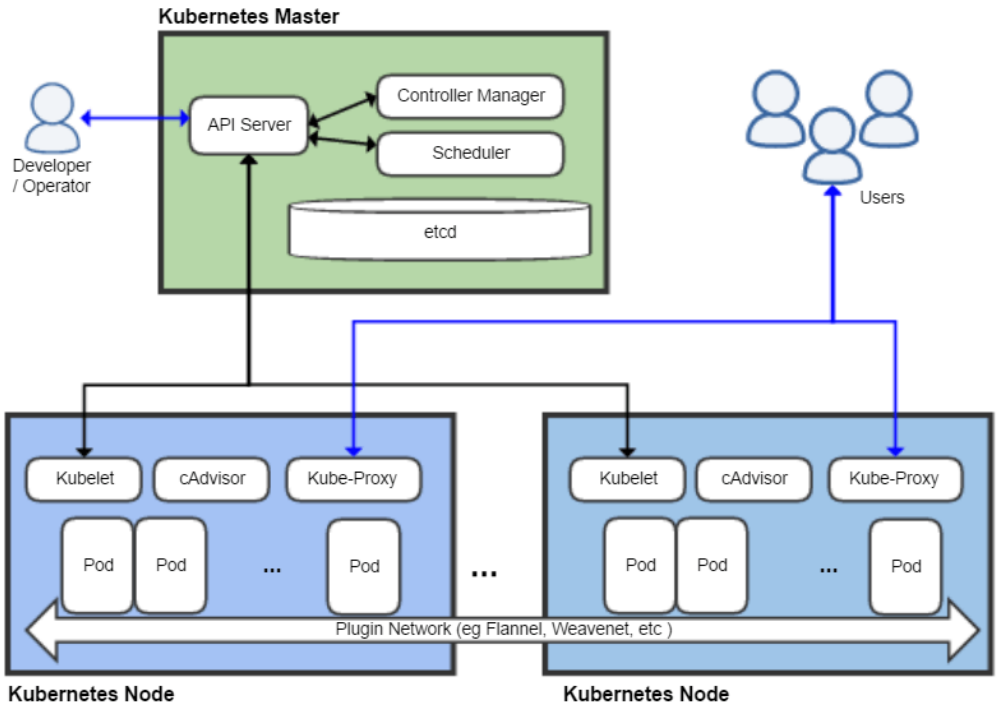

Figure 2.6: Kubernetes Architecture[11].

that have no node assigned, and selects a node for them to run on. Scheduling decisions are based on individual and collective resource requirements, hardware/software/policy constraints, affinity and anti-affinity specifications, data locality, inter-workload interference and deadlines.

  - **kube-controller-manager**: Component on the master that runs controllers. Each controller is logically a separate process, but to reduce complexity, it is compiled into a single binary and run in a single process.

- **Kubernetes Node**: Node components run on each node, maintaining running pods and providing the Kubernetes runtime environment.

  - **kubelet**: It is an agent that runs on each node in the cluster. It ensures that containers are running in a pod. The kubelet takes a set of PodSpecs that are provided through various mechanisms and ensures that the containers described in those PodSpecs are running

and healthy. The kubelet is not responsible for containters that were not created by Kubernetes.

- **kube-proxy** enables the Kubernetes service abstraction by maintaining network rules on the host and performing connection forwarding.
- **Container Runtime**: The container runtime is the software that is responsible for running containers. Kubernetes supports several runtimes such as Docker, containerd, cri-o, rktlet.

2.3.3 gRPC

gRPC (gRPC Remote Procedure Calls) is an open source remote procedure call (RPC) system initially developed at Google[15]. It uses HTTP/2 for transport, Protocol Buffers as the interface description language, and provides features such as authentication, bidirectional streaming and flow control, blocking or non-blocking bindings, and cancellation and timeouts. It generates cross-platform client and server bindings for many languages. Most common usage scenarios include connecting services in microservices style architecture and connect mobile devices, browser clients to backend services.
The main usage scenarios:

- Low latency, highly scalable, distributed systems.
- Developing mobile clients which are communicating to a cloud server.
- Designing a new protocol that needs to be accurate, efficient and language independent.
- Layered design to enable extension eg. authentication, load balancing, logging and monitoring etc.

Steps to create a gRPC service is as follows:

- Define a service in a .proto file.
- Generate server and client code using the protocol buffer compiler.
- Use the gRPC API of the language used to write a simple client and server.

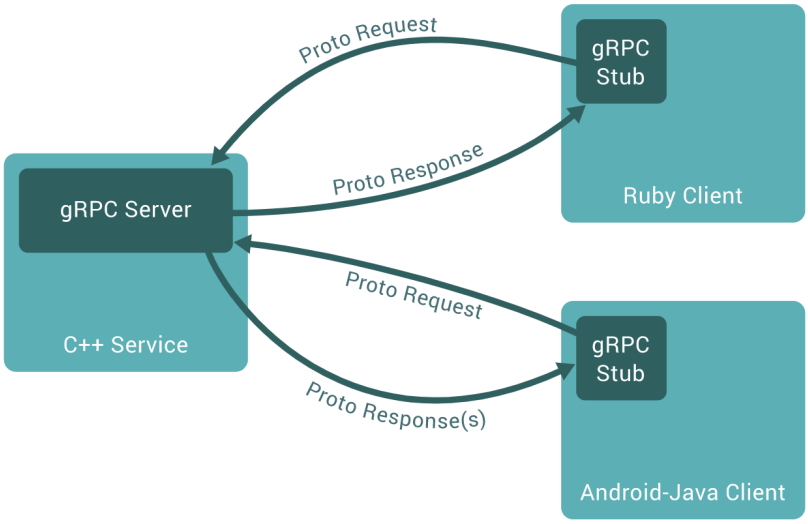

Figure 2.7: gRPC Architecture Diagram.

Protocol Buffers was introduced by Google which is a language-neutral, platform-neutral, extensible mechanism for serialising structured data [27]. By creating a .proto file, a structure for the data is defined, then it uses a special generated source code to easily write and read the structured data to and from a variety of data streams and using a variety of languages. Proto can be updated your data structure without breaking deployed programs that are compiled against the ”old” format.

The example shown is a message format defined in a proto file. A message type has one or more uniquely numbered fields, and each field has a name and a value type. The value types can be numbers, boolean, strings, raw bytes, or even other protocol buffer message types. This allows data to be structured hierarchically.

```
syntax = "proto3";
message MockRequest {
    uint32 latency = 1;
    float error_probability = 2;
    uint32 config = 3;
    uint32 request_counter = 4;
}
```

2.3.4 Service Mesh

A service mesh is a separate infrastructure layer for handling service-to-service communication. It is responsible for the reliable delivery of requests through the complex topology of services for a cloud-native application [33]. In the cloud-native model, a single application might consist of several services which have many instances, and each of those instances might be constantly-changing as an orchestrator like Kubernetes dynamically schedules them. The service communication is complex, part of the run-time behaviour and managing it is vital to ensure performance and reliability. Reliability of request delivery is complex and a service mesh manages this complexity with different techniques such as circuit-breaking, latency-aware load balancing, retries, and deadlines. There are different service meshes such as Istio, Linkerd, Linkerd2, and Consul each of which are built with certain design goals in mind.

Istio Istio is built by Google and IBM in collaboration with Lyft[18]. Istio is an open source framework for connecting, monitoring, and securing microservices. It allows the creation of a network or ”mesh” of deployed services with load balancing, service-to-service authentication, and monitoring, without requiring any changes in service code. Istio provides support to services by deploying an Envoy sidecar proxy to each of the application's pods. The Envoy proxy intercepts all network communication between microservices, and is configured and managed using Istio's control plane functionality.

Linkerd Linkerd is a JVM based network proxy developed by Buoyant which provides a consistent, uniform layer of visibility, reliability, and control across a microservice application. It adds global service instrumentation (latency and success rates, distributed tracing), reliability semantics, (latency-aware load balancing, connection pooling, automatic retries and circuit breaking), and security features such as transparent TLS encryption.

Linkerd2 Linkerd2 is a rewrite of Linkerd in Go with the intention to move away from using JVM, make it lightweight and focused on integration with

Kubernetes. There are continuous developments and there are certain features around reliability, security and traffic shifting that are under development.

Consul Consul is a control plane built by HashiCorp that works with multi-datacenter topologies and specialises in service discovery. Consul works with many data planes and can be used with or without other control planes such as Istio.

3 Implementation

This chapter describes the implementation of the setup that was used to test the resiliency of Envoy proxies deployed as sidecars.

3.1 Services Setup

A collection of three microservices each with their sidecar proxies are deployed in two different configurations. Each of the configurations is load tested with different parameters. The service communicates with each other through gRPC. The intention was to find the most reoccurring configurations of microservices in a large scale system and perform experiments on them to understand the effects of Envoy. Following were the two common configurations found:

3.1.1 Configuration 1

In this configuration as seen in figure 3.1, the client makes a request call to service 1 which calls two upstream services 2 and 3 simultaneously. Service 1 returns a response only after receiving responses from the two upstream services. Each of the services are deployed with a sidecar proxy which handles the communication between the services.

3.1.2 Configuration 2

In this configuration as seen in figure 3.2, the client makes a request call to service 1 which calls service 2 and service 2 in turn makes a service call to service 3. Service 1 sends a response after it receives a response from service 2, which only sends a response once it has received a response from service 3. Each of the services are deployed with a sidecar proxy which handles the communication between the services.

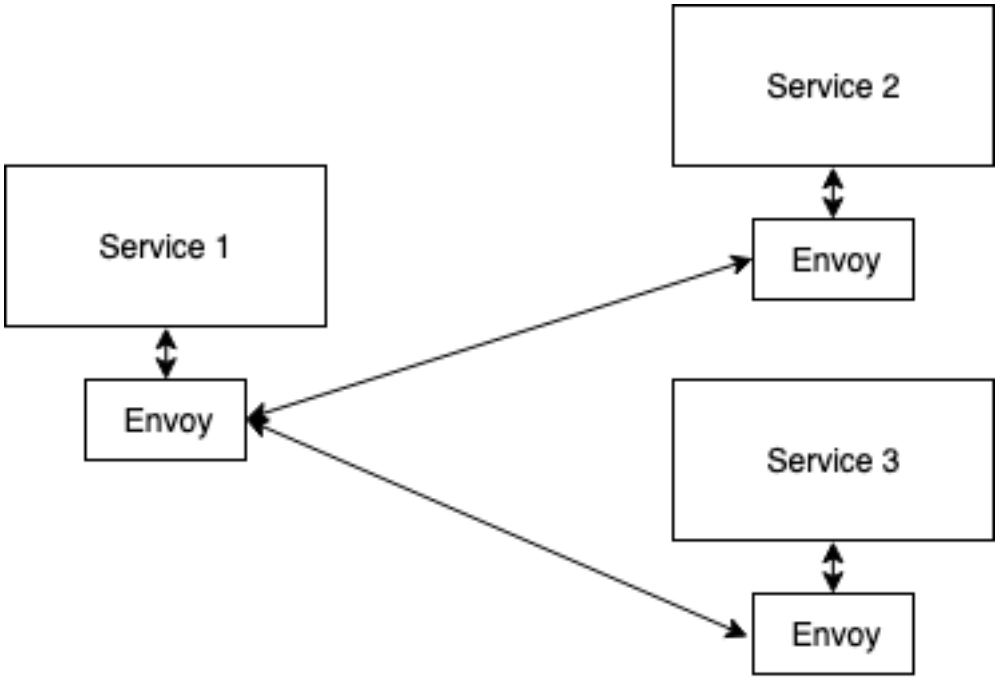

Figure 3.1: Configuration 1.

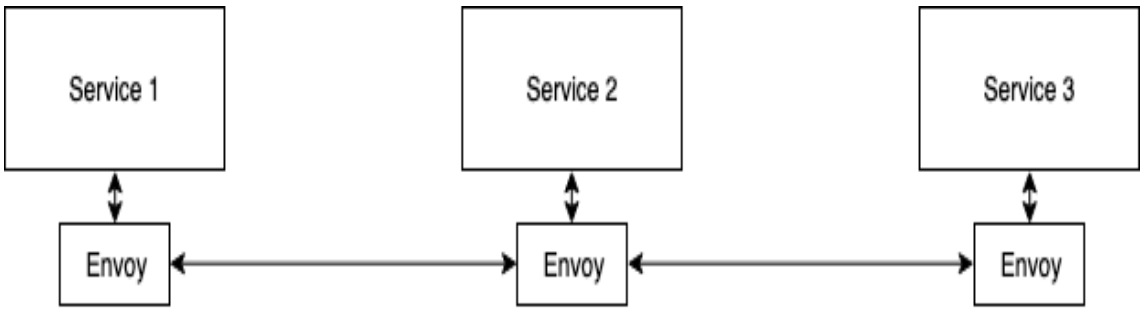

Figure 3.2: Configuration 2.

3.2 Apollo Services

Apollo is a set of open-source Java libraries used for writing micro-services[1]. Apollo includes features such as an HTTP server and a URI routing system, making it trivial to implement RESTful services.

Apollo has three main components:

- **apollo-api**: The apollo-api library defines the interfaces for request routing and request/reply handlers.
- **apollo-core**: The apollo-core library manages the lifecycle (loading,

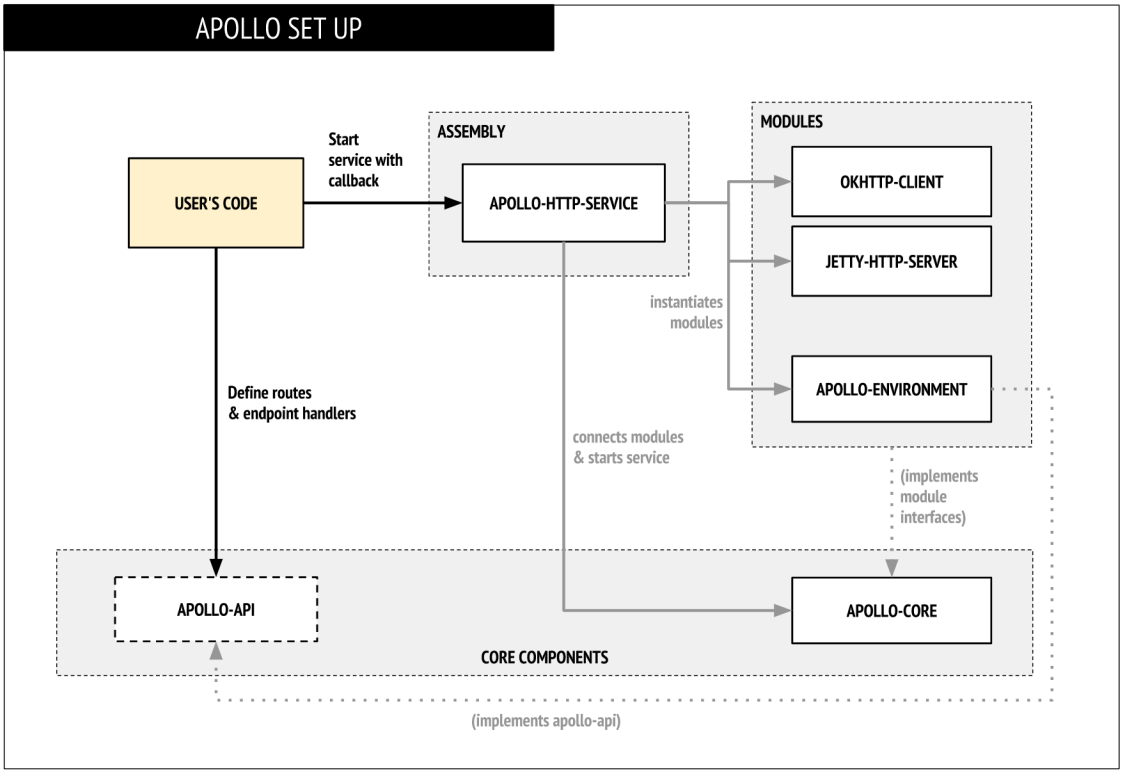

Figure 3.3: Apollo Setup.

starting, and stopping) of the service and defines a module system for adding functionality to an Apollo assembly.

- **apollo-http-service**: The apollo-http-service library is a standardized assembly of Apollo modules. It incorporates both apollo-api and apollo-core and ties them together with several other modules to get a standard api service using HTTP for incoming and outgoing communication.

As seen in figure 3.3, when the user's code starts a service with callback, the apollo-http-service instantiates different modules such as okhttp-client, jetty-http-server, apollo-environment. It then connects the different modules and starts the service by calling the apollo-core component.

3.3 Load Testing Client

3.3.1 ghz

A tool called *ghz* is used in the experiment to make the service requests.[2] It is a command line utility and Go package for load testing and benchmarking gRPC services. It is can be used for testing and debugging services locally, and in automated continuous integration environments for performance regression testing.
Additionally the core of the command line tool is implemented as a Go library package that can be used to programatically implement performance tests as well.

```
$ ./ghz --insecure --proto ./mock.proto --call Service.Mock
-d '{"latency" :10, "error_probability" : 0.0035, "config":1}'
-n 10000 0.0.0.0:5990
```

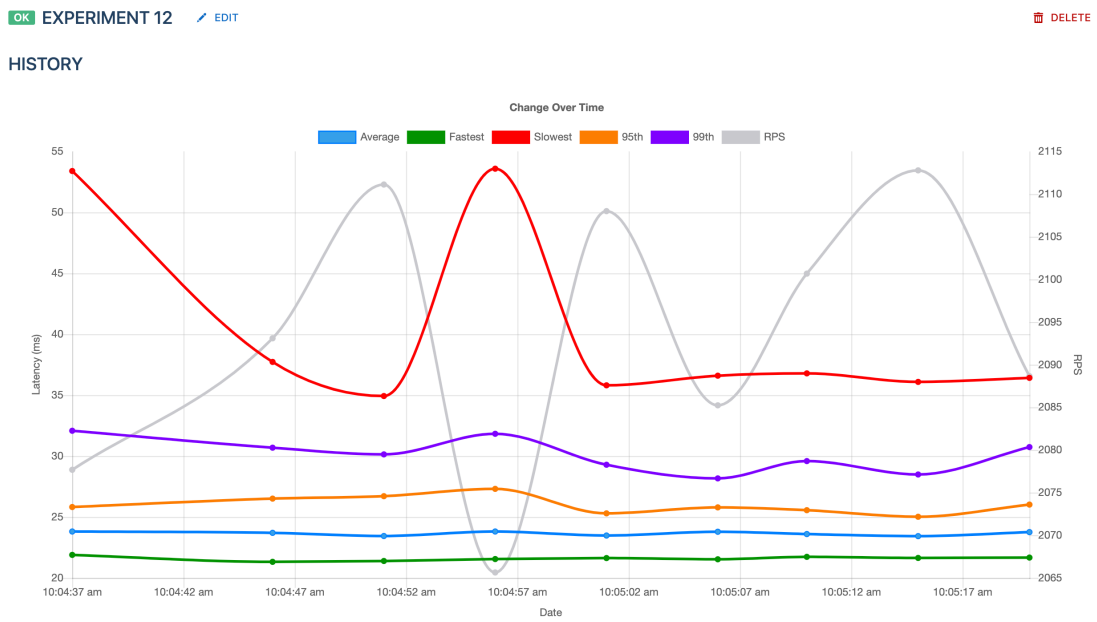

Figure 3.4: View of *ghz-web* showing the history of experiment runs.

3.3.2 ghz-web

ghz-web is a web server and a web application for storing, viewing and comparing data that has been generated by the *ghz* CLI. [14] Figure 3.4 shows the trends

of average time taken by each request in the experiments along with the fastest request, slowest, requests per second, time taken by 95% of requests and the time taken by 99% of requests.

3.4 Deployed Environment

The experiments were run on a shared Virtual Machine instance in Google Cloud Platform with 32 core Intel Xeon E5 v3 (Haswell) CPUs of and 120 GB of RAM.

3.5 Experiment Design

Using the setup of services as shown in figures 3.1 and 3.2, we perform load testing using *ghz*. Each microservice can be configured to have a certain error probability and latency. Using Java CompletableFuture [5], the requests are handled asynchronously. The latency set (in milliseconds) as a parameter in the request is handled by *CompletableFuture* asynchronously and will have the new thread sleep in the background. For the purposes of the experiment we set the latency of each service at 10ms and have different error probability. The data from each experiment without using Envoy as a sidecar proxy is compared with the data from experiments where the microservices use a sidecar proxy. The different error probabilities chosen used to run experiments are, 0%, 0.0035%, 0.02%, 0.2%, 2%, and 20%.

The error probability of 0% was chosen to test the setups in an ideal scenario. 0.0035% is an acceptable error reply percentage for a large scale production grade system. 10 fold increments from 0.02% to 20% of error probability were chosen to understand how the systems performed with and without Envoy and its resiliency patterns.

4 Results and Evaluation

4.1 Results

This chapter contains the results of the experiments conducted. The chapter is further divided into sections, the first section contains the results of experiments run without Envoy service proxy, and the second section contains the results from the experiments run with Envoy service proxy. This chapter mainly contains the results and preliminary analysis. A detailed analysis is provided in the next chapter.

For the purposes of the experiments, the latency for each microservice was set at 10ms and the error probability was increased to observe the effects on the system. Experiments were performed using the load testing tool *ghz* by sending 10000 requests on each run with an error probability of 0% to test the system in an ideal scenario without any errors to see the average time taken per requests and throughput. These experiments were repeated ten times. Increasing the error probability to 0.0035% and then 0.02% which is the error probability of a production grade large scale system. At these error probabilities, the resilience of Envoy proxies are tested but are not pushed to its limit. By increasing the error probabilities to 0.2%, 2%, and finally to 20% we can test the Apollo services, gRPC and observe how they are able to handle failures. Additionally, the resilience of Envoy is pushed to its limit. We were able to observe the way Envoy uses automatic retries and circuit breaking to limit the number of errors.

4.1.1 Experiments without Envoy

The experiments are first run without using Envoy in both configurations as shown in figures 3.1 and 3.2. The values of time in the table are in milliseconds. P95 and P99 refer to the time taken by 95% and 99% of the requests respectively.

As seen from table 4.1 there is no significant difference in the time taken by the different configurations with the same parameters.

4.1.2 Experiments with Envoy

The following table is the data obtained by performing load testing on configurations with Envoy deployed as a sidecar proxy.

Error Probability		No Envoy	Envoy
0.2	Fastest Slowest P95 P99	22 35.81 24.26 25.58	23.1 1010 116.96 240.16
0.02	Fastest Slowest P95 P99	21.88 35.11 24.57 26.16	22.63 124.69 38.48 62.24
0.002	Fastest Slowest P95 P99	21.69 35.64 24.55 26.77	22.24 76 29.93 37.46
0.0002	Fastest Slowest P95 P99	21.83 36.52 25.28 28.36	22.02 65.18 25.1 32.15
0.000035	Fastest Slowest P95 P99	21.9 36.71 26.18 30.85	23.12 39.81 29.33 34.13
0	Fastest Slowest P95 P99	21.64 36.09 25.02 28.49	22.22 35.54 26.72 31.04

Table 4.1: Comparison of time taken by requests in configuration 1 with and without using Envoy (in milliseconds)

4.1.3 Comparison of execution times

Table 4.1 contains comparison data of experiments run with and without Envoy in configuration 1 and table 4.2 contains the comparison of time taken by experiments with and without Envoy in configuration 2.

As seen in table 4.2, the time taken by each request in the experiments without Envoy is faster than the experiments that use Envoy. When the error probability

Error Probability		No Envoy	Envoy
0.2	fastest	21.7	22.83
	slowest	47.65	1390
	p95	24.55	124.15
	p99	27	257.27
0.02	fastest	21.97	22.7
	slowest	33.87	154.36
	p95	24.25	41.69
	p99	25.55	59.57
0.002	fastest	21.62	22.7
	slowest	36.12	154.36
	p95	24.92	41.69
	p99	27.41	59.57
0.0002	fastest	21.68	22.33
	slowest	37.73	75.11
	p95	25.42	29.67
	p99	29	35.44
0.000035	fastest	21.8	21.94
	slowest	37.61	64.63
	p95	26.57	25.54
	p99	30.62	30.23
0	fastest	21.67	22.15
	slowest	36.42	35.69
	p95	26.02	26.66
	p99	30.74	31.66

Table 4.2: Comparison of time taken by requests in configuration 2 with and without using Envoy (in milliseconds)

reduces, the values of P95 and P99 for experiments with and without Envoy converge. In an ideal scenario of 0% error probability, the time taken by experiments using Envoy on average is 3.42% slower than the experiments without using Envoy.

4.2 Evaluation

The evaluation of the results will answer the following questions:

- Is Envoy good to be used in production environments?
- Does the benefits of Envoy outweigh the disadvantages?

4.2.1 Envoy in Production

The results in table 4.1 show that average time taken by services using Envoy is slightly higher than those which do not use Envoy as a sidecar proxy. Each of the services were set at certain error probability while performing load testing, when each service has a certain error probability, the error probability of the entire system is compounded. For example, if the error probability each service is 0.2, the compounded error probability of the entire system with three services is

$$(1 - (1 - 0.2) * (1 - 0.2) * (1 - 0.2)) = 0.488$$

This means that about 48.8% of all requests will be errors. Figure 4.1 depicts the distribution of errors for an experiment with each service having a error probability of 0.2.

When the error probability of the service reduces, as shown in table 4.3, the error probability of the entire system reduces too. When there are high number of errors, the throughput is hindered severely. When there are many requests failing there is a need to have circuit breakers in place to make sure that these errors do not affect the entire system's performance. This is done by throttling the number of incoming requests and allowing the system to recover from the errors.

Figure 4.1: Error Distribution without Envoy.

When testing services with Envoy the throughput i.e., requests per second (RPS) was higher for services without Envoy for error probabilities such as

Error Probability of each service	Error Probability of the system
0.2	0.488
0.02	0.058808
0.002	0.005988008
0.0002	0.000599880008
0.000035	0.000104996325
0	0

Table 4.3: Compounded Error Probability of the entire system for the corresponding error probability of each service

0%, 0.0035%, and 0.02%. However, the RPS declined rapidly for higher error probabilities. Figure 4.2 shows the trend in which RPS reduced for higher error probabilities.

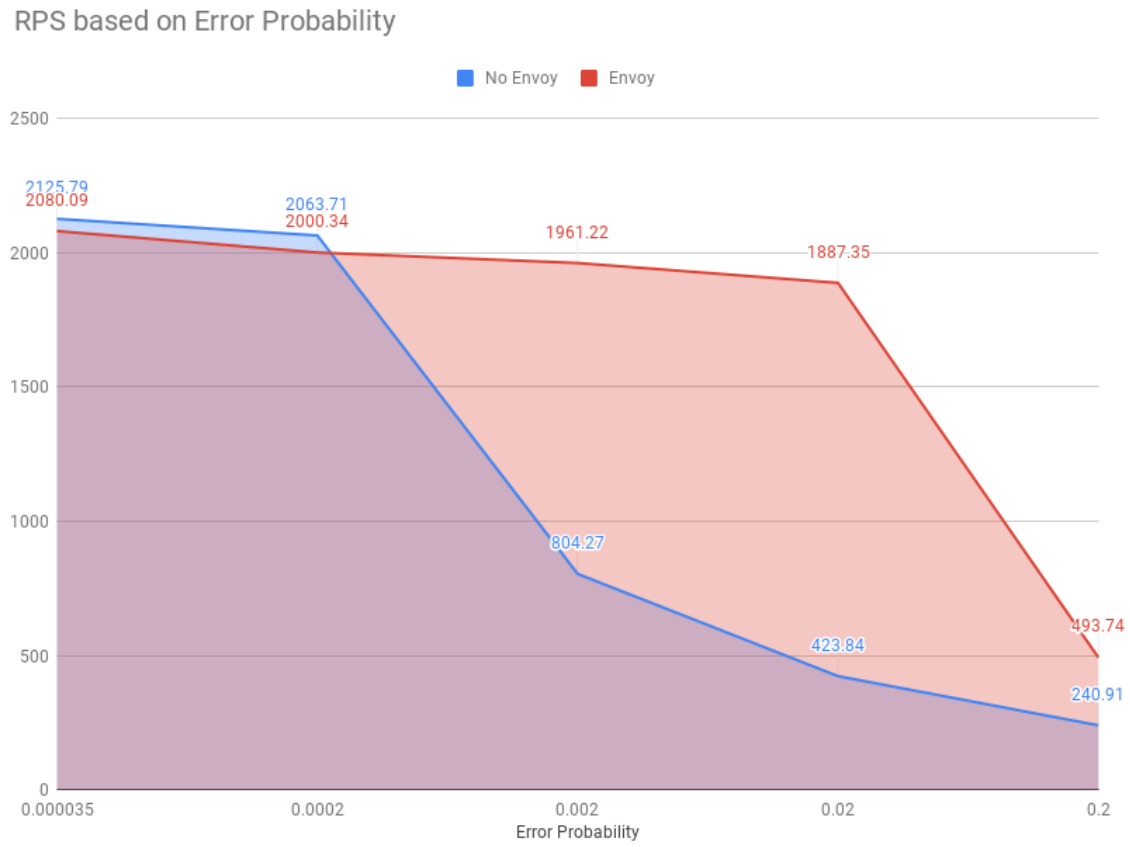

Figure 4.2: RPS v/s Error Probability.

4.3 Envoy's Resilience Capabilities

When Envoy was deployed as a sidecar proxy, the proxy employs features such as automatic retries, rate limiting and circuit breakers to minimise the number of errors thrown by the system. As seen from figure 4.3, the number of errors thrown by the services with Envoy are lower than the number of error thrown by the services which do not use Envoy. This reduces the percentage of overall errors at 20% error probability to 13.8% as compared to 48.8% when not using Envoy. Having higher error percentages decreases the RPS as shown in figure 4.2.

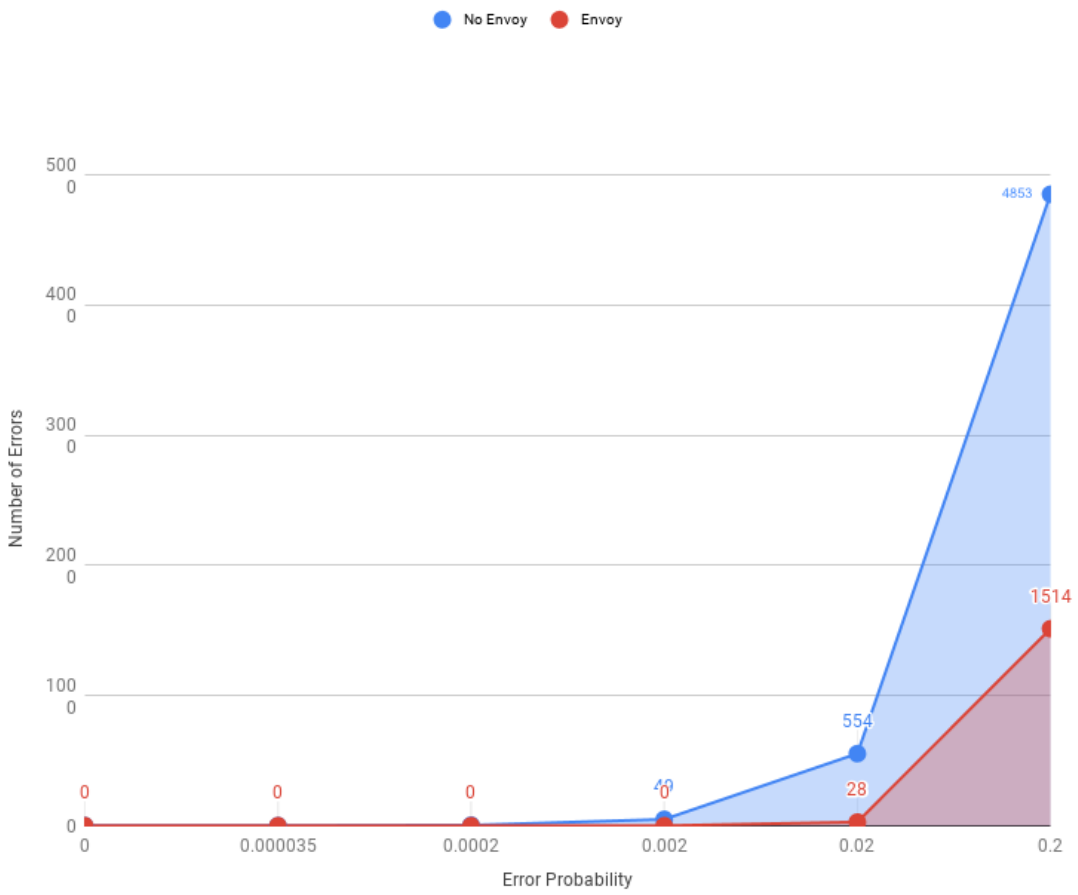

Figure 4.3: Error Probability v/s Number of Errors.

4.3.1 Data from Envoy Proxies

When running experiments with the Envoy service proxies deployed as sidecars, we were able to capture the number of failed requests, the number of requests

that were retried, number of requests that were not retried by each sidecar from the proxies' logs. These provide insights on the resilience offered by the service proxies. The logs from from an experiment which had a error probability of 2% retrieved from each of the Envoy sidecar proxies is present in Appendix A.

The `upstream_rq_200` log is the total number of requests the Envoy proxy retried and received a HTTP 200 response. `upstream_rq_completed` provides the total number of upstream requests that are completed. `retry_or_shadow_abandoned` is the total number of times shadowing or retry buffering was cancelled due to buffer limits. This happens when there are too many requests are failing and the proxy cannot perform retries on all of them due to limits on resources. `upstream_rq_retry_overflow` is the total requests not retried due to circuit breaking. This is when the circuit breaker is set to OPEN state to try and provide the service time to self-correct. `upstream_rq_retry_success` is the total request retry successes.

As seen in figure 4.2, due to the automatic retry capability of Envoy the requests per second in the experiments run with Envoy are much higher than the one run without Envoy. This is because Envoy retries failed requests, which increases the throughput and the total time taken to complete the experiment. The total time taken by the experiment to send all requests determines the requests per second for the experiment.

5 Conclusions

With the adoption of microservices, it has been clear that even though they reduce complexity in the services itself, they add a lot of operational complexity. To improve availability, resilience and scalability - tooling such as Kubernetes, Envoy proxy, Istio and others have been developed. Envoy service proxy deployed as a sidecar to microservices provide the ability to have higher resilience, advanced load balancing and distributed tracing. In this thesis, we determined in the context of our testing setup if Envoy adds latency to the system and if the disadvantages outweighed its advantages. After conducting the experiments and analysing the results, we can conclude that the Envoy service adds very little latency to the request time. Although the presence of Envoy adds latency into the system, with the data obtained from the experiments, we can say that the latency was not statistically significant. We performed Students T-distribution [17] between the time taken by both configurations with and without Envoy. Additionally, the resilience offered by Envoy service proxy improves the overall performance of the system making it more resilient to failures. The advantages of having an Envoy sidecar proxy based on the experiment setup, the data gathered and the analysis made outweigh the disadvantage it brings in the form of additional latency.

5.1 Discussion

The goals initially set when testing Envoy was to see if it added a lot of latency to the system and observe the resilience features of it. When performing load testing on very low error probabilities such as 0.002%, 0.035% and 0.2% of each service, the sidecar proxy performed as expected by retrying all the failed requests and reducing the system's error percentages to zero. However, on very high error probabilities such as 20% on each service, there were just too many requests that were failing, and even with the Envoy proxies, all the failures could not be handled. There is a lesser number of failures compared to the system without Envoy; this is due to Envoy's automatic retries, rate limiting, and circuit breaking features. While trying to disable these features to see if it made any difference in request

times, we found that Envoy's functions cannot be truly disabled, and by design, it is meant to be present. The possible workaround was to increase the value for maximum connections and maximum requests to a very high number. The configurations used for Envoy is present in Appendix B. Similarly, by reducing the value for the number of retries to zero, the automatic retries feature could be disabled.

5.2 Future Work

Service Proxies and Service Meshes are very young and fast developing. There are additional features added to make the systems more resilient, scalable and available. Envoy service proxies are used as the data plane in Istio service mesh. The control plane provides additional features such as telemetry, traffic routing and policy enforcement. These are features that help a system in a microservices architecture not only to grow in scale but assist in maintaining and monitoring of the system. Introducing Kubernetes and service meshes into the experiments would give insights into the effects these systems might have on each other and the performance of the overall system. Our experiments consisted of three services having Envoy proxies as sidecars, however, in a large scale system there are dozens if not hundreds of services and performing testing on a setup with hundreds of services would give us more accurate data about the advantages of Envoy.

References

[1] *Apollo*. http://spotify.github.io/apollo/. (Accessed on 06/10/2019).

[2] *bojand/ghz: Simple gRPC benchmarking and load testing tool*. https://github.com/bojand/ghz. (Accessed on 06/10/2019).

[3] Buzen, J. P. and Gagliardi, U. O. "The Evolution of Virtual Machine Architecture". In: *Proceedings of the June 4-8, 1973, National Computer Conference and Exposition*. AFIPS '73. New York, New York: ACM, 1973, pp. 291–299. DOI: 10.1145/1499586.1499667. URL: http://doi.acm.org/10.1145/1499586.1499667.

[4] Chen, Lianping. "Microservices: Architecting for Continuous Delivery and DevOps". In: Mar. 2018. DOI: 10.1109/ICSA.2018.00013.

[5] *CompletableFuture (Java Platform SE 8)*. https://docs.oracle.com/javase/8/docs/api/java/util/concurrent/CompletableFuture.html. (Accessed on 06/27/2019).

[6] *Computer Network | Layers of OSI Model - GeeksforGeeks*. https://www.geeksforgeeks.org/layers-osi-model/. (Accessed on 06/20/2019).

[7] *Consul by HashiCorp*. https://www.consul.io/. (Accessed on 06/10/2019).

[8] *Elastic Load Balancing - Amazon Web Services*. https://aws.amazon.com/elasticloadbalancing/. (Accessed on 06/10/2019).

[9] *Elastic Load Balancing - Amazon Web Services*. https://aws.amazon.com/elasticloadbalancing/. (Accessed on 06/11/2019).

[10] *Envoy Proxy - Home*. https://www.envoyproxy.io/. (Accessed on 06/06/2019).

[11] *File:Kubernetes.png - Wikimedia Commons*. https://commons.wikimedia.org/w/index.php?curid=53571935. (Accessed on 06/10/2019).

[12] *Finagle*. https://twitter.github.io/finagle/. (Accessed on 06/10/2019).

[13] *Get Started, Part 1: Orientation and setup | Docker Documentation*. https://docs.docker.com/get-started/. (Accessed on 06/10/2019).

[14] *ghz-web*. https://ghz.sh/docs/web/intro. (Accessed on 06/10/2019).

[15] *gRPC*. https://grpc.io/. (Accessed on 06/06/2019).

[16] *HAProxy - The Reliable, High Performance TCP/HTTP Load Balancer*. http://www.haproxy.org/. (Accessed on 06/10/2019).

[17] Hill, G. W. "Algorithm 395: Students T-distribution". In: *Commun. ACM* 13.10 (Oct. 1970), pp. 617–619. ISSN: 0001-0782. DOI: 10.1145/355598.362775. URL: http://doi.acm.org/10.1145/355598.362775.

[18] *Istio*. https://istio.io/. (Accessed on 06/06/2019).

[19] Krafzig, Dirk, Banke, Karl, and Slama, Dirk. *Enterprise SOA: Service-Oriented Architecture Best Practices (The Coad Series)*. Upper Saddle River, NJ, USA: Prentice Hall PTR, 2004. ISBN: 0131465759.

[20] *Learn About the Microservices Architecture*. https://docs.oracle.com/en/solutions/learn-architect-microservice/index.html#GUID-BDCEFE30-C883-45D5-B2E6-325C241388A5. (Accessed on 06/10/2019).

[21] *Linkerd - Introduction*. https://linkerd.io/1/overview/. (Accessed on 06/10/2019).

[22] *Linkerd - Overview*. https://linkerd.io/2/overview/. (Accessed on 06/10/2019).

[23] *Netflix/Hystrix: Hystrix is a latency and fault tolerance library designed to isolate points of access to remote systems, services and 3rd party libraries, stop cascading failure and enable resilience in complex distributed systems where failure is inevitable*. https://github.com/Netflix/Hystrix. (Accessed on 06/10/2019).

[24] *NGINX | High Performance Load Balancer, Web Server, & Reverse Proxy*. https://www.nginx.com/. (Accessed on 06/10/2019).

[25] *(PDF) Service-Oriented Architecture and Software Architectural Pattern – A Literature Review | Kholed Langsari - Academia.edu*. https://www.academia.edu/24716210/Service-Oriented_Architecture_and_Software_Architectural_Pattern_A_Literature_Review. (Accessed on 06/11/2019).

[26] *Production-Grade Container Orchestration - Kubernetes*. https://kubernetes.io/. (Accessed on 06/10/2019).

[27] *Protocol Buffers | Google Developers*. https://developers.google.com/protocol-buffers/. (Accessed on 06/10/2019).

[28] *Retry pattern - Cloud Design Patterns | Microsoft Docs*. https://docs.microsoft.com/en-us/azure/architecture/patterns/retry. (Accessed on 06/10/2019).

[29] *SmartStack: Service Discovery in the Cloud – Airbnb Engineering & Data Science – Medium*. https://medium.com/airbnb-engineering/smartstack-service-discovery-in-the-cloud-4b8a080de619. (Accessed on 06/10/2019).

[30] Smith, J.E. and Nair, Ravi. "The architecture of virtual machines". In: *Computer* 38.5 (May 2005), pp. 32–38. DOI: 10.1109/mc.2005.173. URL: https://doi.org/10.1109/mc.2005.173.

[31] *Using a Three-Tier Architecture Model - Windows applications | Microsoft Docs*. https://docs.microsoft.com/en-us/windows/desktop/cossdk/using-a-three-tier-architecture-model. (Accessed on 06/11/2019).

[32] Verma, Abhishek et al. "Large-scale cluster management at Google with Borg". In: *Proceedings of the European Conference on Computer Systems (EuroSys)*. Bordeaux, France, 2015.

[33] *What's a service mesh? And why do I need one? | Buoyant*. https://buoyant.io/2017/04/25/whats-a-service-mesh-and-why-do-i-need-one/. (Accessed on 06/10/2019).

[34] *White Paper: The Definitive Guide to Container Platforms | Docker*. https://www.docker.com/resources/white-paper/the-definitive-guide-to-container-platforms. (Accessed on 06/10/2019).

Appendices

Appendix - Contents

A Envoy Logs

```
ENVOY 1
cluster.envoytest.retry.upstream_rq_200: 417
cluster.envoytest.retry.upstream_rq_2xx: 417
cluster.envoytest.retry.upstream_rq_completed: 417
cluster.envoytest.retry_or_shadow_abandoned: 0
cluster.envoytest.upstream_rq_retry: 417
cluster.envoytest.upstream_rq_retry_overflow: 38
cluster.envoytest.upstream_rq_retry_success: 408
ENVOY 2
cluster.envoytest.retry.upstream_rq_200: 396
cluster.envoytest.retry.upstream_rq_2xx: 396
cluster.envoytest.retry.upstream_rq_completed: 396
cluster.envoytest.retry_or_shadow_abandoned: 0
cluster.envoytest.upstream_rq_retry: 396
cluster.envoytest.upstream_rq_retry_overflow: 15
cluster.envoytest.upstream_rq_retry_success: 386
ENVOY 3
cluster.envoytest.retry.upstream_rq_200: 413
cluster.envoytest.retry.upstream_rq_2xx: 413
cluster.envoytest.retry.upstream_rq_completed: 413
cluster.envoytest.retry_or_shadow_abandoned: 0
cluster.envoytest.upstream_rq_retry: 413
cluster.envoytest.upstream_rq_retry_overflow: 13
cluster.envoytest.upstream_rq_retry_success: 405
```

B Envoy Configuration

```
admin:
  access_log_path: /tmp/admin_access.log
  address:
    socket_address:
      protocol: TCP
      address: 127.0.0.1
      port_value: 9901
static_resources:
  listeners:
  - name: listener_0
    address:
      socket_address:
        protocol: TCP
        address: 0.0.0.0
        port_value: 5991
    filter_chains:
    - filters:
      - name: envoy.http_connection_manager
        config:
          codec_type: auto
          stat_prefix: ingress_http
          route_config:
            name: local_route
            virtual_hosts:
            - name: local_service
              domains: ["*"]
              routes:
              - match:
                  prefix: "/"
                  grpc: {}
                route:
```

```
                host_rewrite: 10.48.32.124
                cluster: envoytest
                retry_policy:
                  retry_on: cancelled,deadline-exceeded,internal,
                            resource-exhausted,unavailable
                  num_retries: 10

        http_filters:
        - name: envoy.router
clusters:
- name: envoytest
  connect_timeout: 0.25s
  type: LOGICAL_DNS
  dns_lookup_family: V4_ONLY
  lb_policy: ROUND_ROBIN
  http2_protocol_options: {}
  circuit_breakers:
    thresholds:
      priority: HIGH
      max_connections: 10000
      max_pending_requests: 10000
      max_requests: 10000
      max_retries: 10000
  hosts:
    - socket_address:
        address: 10.48.32.124
        port_value: 5990
```

TRITA-EECS-EX-2019:226